

# Parallel Text Search

Zwick, Patrick

patrick.zwick@htwg-konstanz.de

Döbele, Max

max.doebele@htwg-konstanz.de

Friesen, Marcel

marcel.friesen@htwg-konstanz.de

06 February 2026

## Abstract

## 1 Introduction

## 2 Testing Data

For testing, we use a collection of up to 98 books from Project Gutenberg, which we concatenate into a single text containing over 82 million characters. We perform queries using, at most, the 100 most common English words. Both the number of books and the number of queries can be varied to evaluate performance under different conditions.

## 3 Sequential Implementations

Goals of the sequential implementations: Simple, easy to parallize, as few dependencies as possible

### 3.1 Reference Implementation

To verify correctness, we use a simple baseline implementation, denoted `std`, which relies on `std::string::find()` for substring matching (see Listing 1).

```
1 std::vector<std::vector<size_t>>
2 find_std(const std::string &text, const std::vector<std::string> &queries)
3 {
4     std::vector<std::vector<size_t>> indices(queries.size());
5     for (size_t i = 0; i < queries.size(); ++i) {
6         const std::string &query = queries[i];
7         auto pos = text.find(query);
8         while (pos != std::string::npos) {
9             indices[i].push_back(pos);
10            pos = text.find(query, pos + 1);
11        }
12    }
13    return indices;
14 }
```

Listing 1: `find_std()` in `std_text_search.cpp`

### 3.2 Candidate Selection and Validation Approach

The first implementation is divided into two distinct phases:

1. Identification of candidate positions for potential occurrences based on a simple filtering criterion. A position is considered a candidate if three characters match: the first character of the query, the middle character, and the last character.

2. Verification of each candidate position to determine whether it constitutes a valid occurrence.

The initial approach, denoted `candidate_v1`, performs candidate identification followed by validation separately for each query (see Listing 2). The vector is cleared after each query and reused for the next, avoiding unnecessary allocations.

```
1 void find_candidates(const size_t text_length, std::vector<size_t>
    &results,
2                     const std::string &text, const std::string &query) {
3     const auto query_length = query.length();
4     const auto mid = query_length >> 1;
5     const auto end = query_length - 1;
6     for (size_t i = 0; i < text_length - query_length; ++i) {
7         if (text[i] == query[0] && text[i + mid] == query[mid] &&
8             text[i + end] == query[end]) {
9             results.push_back(i);
10        }
11    }
12 }
13
14 bool test_candidate(const size_t index, const std::string &text,
15                    const std::string &query) {
16     return std::memcmp(text.data() + index, query.data(), query.size()) ==
17        0;
18 }
19
20 std::vector<std::vector<size_t>>
21 find_candidate_v1(const std::string &text,
22                  const std::vector<std::string> &queries) {
23     std::vector<std::vector<size_t>> indices(queries.size());
24     const auto text_length = text.length();
25     std::vector<size_t> results;
26     for (size_t i = 0; i < queries.size(); ++i) {
27         const std::string &query = queries[i];
28         find_candidates(text_length, results, text, query);
29         for (const auto &result : results) {
30             if (test_candidate(result, text, query)) {
31                 indices[i].push_back(result);
32             }
33         }
34         std::memset(results.data(), 0, results.size() * sizeof(size_t));
35     }
36     return indices;
37 }
```

Listing 2: `find_candidates()`, `test_candidate()` and `find_candidate_v1()` in `candidate_v1_text_search.cpp`

### 3.2.1 Using an Array Instead of `std::vector`

Since a `std::vector` may need to repeatedly grow its underlying storage and copy existing elements when its capacity is exceeded, this can introduce a significant performance bottleneck, at least for the first query. To avoid this overhead, the `candidate_v2` implementation instead pre-allocates an array with a size equal to the length of the text. In the worst case, every position in the text could be a valid candidate, so this allocation is sufficient to store all candidates.

### 3.2.2 Using a Bitmask

To improve CPU cache utilization and reduce the memory footprint, the `candidate_v3` implementation represents the candidate buffer as a *bitmask*, where each bit set to 1 denotes a valid candidate position. The bitmask is stored as an array of unsigned 64-bit integers, allowing 64 candidates to be packed into a single word.

Let  $m$  denote the length of the text. To represent candidate positions for a query of any length, we require

$$h = \left\lceil \frac{m + 63}{64} \right\rceil$$

64-bit words in the bitmask.

For a candidate at position  $k$ , the word index  $w$  of the corresponding 64-bit word  $B_w$  in the bitmask is obtained by dividing  $k$  by 64 which can be efficiently computed by a right shift of 6 bits:

$$w = \frac{k}{64} = k \gg 6.$$

The bit position  $b$  within the word  $B_w$  is given by taking  $k$  modulo 64, which can be efficiently computed by a bitwise AND with 63:

$$b = k \bmod 64 = k \& 63.$$

To set this bit, we construct a temporary value with only the target bit active by shifting the integer constant 1 to the left by the bit position  $b$ , and then update the mask word using the bitwise OR:

$$B_w \leftarrow B_w \mid (1 \ll b).$$

To reconstruct candidate positions from the bitmask, we iterate over all 64-bit words and extract the indices of the set bits.

For a given word index  $w$ , let  $B_w$  again denote the corresponding 64-bit word. If  $B_w = 0$ , the word contains no valid candidates. Otherwise, we repeatedly extract the least significant set bit until  $B_w$  becomes zero.

The position of this bit within the word is obtained using the number of trailing zeros:

$$b = \text{ctz}(B_w),$$

where  $\text{ctz}$  denotes the count of trailing zero bits.

The corresponding global candidate index is then

$$k = 64w + b.$$

After processing a candidate, the lowest set bit is cleared using the operation

$$B_w \leftarrow B_w \& (B_w - 1),$$

which removes the extracted bit and guarantees termination after all set bits have been processed.

This approach ensures that candidate extraction runs in time proportional to the number of valid candidates rather than the size of the bitmask.

### 3.2.3 Using a Bitmask for All Queries Together

Instead of allocating a separate bitmask for each query and clearing it after processing, the `candidate_v4` implementation uses single bitmask large enough to store candidate positions for *all queries* simultaneously.

For  $n$  queries, the total bitmask size becomes

$$h_{\text{total}} = n \cdot h,$$

where  $h$  is the number of 64-bit words needed for a single query.

To access the word corresponding to a candidate at position  $k$  for query  $q$ , we first compute the offset for that query:

$$l = q \cdot h.$$

The global word index in the combined bitmask is then

$$w_{\text{total}} = l + (k \gg 6),$$

where, as before,  $k \gg 6$  gives the word index within the query's portion of the bitmask.

Reconstructing candidate positions proceeds in a similar manner. We iterate over all queries  $q$ , and within each query, over all words  $w$  from 0 to  $h - 1$ . For each word index, the offset-adjusted index is

$$w_{\text{total}} = l + w.$$

Candidate positions are then extracted from the set bits of  $B_{w_{\text{total}}}$  using the same approach described previously.

### 3.3 Rolling Hash Approach Rabin-Karp Algorithm

Assume we are searching for the word "hello" in a large text with millions of characters. Using a naive approach, one would have to check up to all 5 characters for every single position in the text. This results in a time complexity of:

$$O(N \cdot M)$$

where:

- $N$  = Length of the text
- $M$  = Length of the searched word (query)

If we search for  $P$  different words, the complexity multiplies by the number of queries, resulting in  $O(N \cdot M \cdot P)$ . To avoid comparing every single character for every shift, we convert the search terms into a numerical value, known as a **hash value**:

$$\text{query\_hash} = q_0 \cdot B^{M-1} + q_1 \cdot B^{M-2} + \dots + q_{M-1} \cdot B^0$$

where  $B$  is a large prime number (PRIME) acting as the base, and  $q_i$  is the numerical value of the character.

To ensure we do not have to recalculate the hash value from scratch for every position in the text, we use the **Rabin-Karp Rolling Hash** algorithm [1]:

1. **Subtract** the most significant character:

```
1 window_hash -= static_cast<uint64_t>(static_cast<uint8_t>(text[j  
- 1])) * power;
```

2. **Shift** the window:

```
1 window_hash *= PRIME;
```

3. **Add** the new character:

```
1 window_hash += static_cast<uint8_t>(text[j + m - 1]);
```

Finally, we handle potential **hash collisions** (where different strings produce the same hash due to the  $\text{mod } 2^{64}$  nature of `uint64_t`) by verifying the match:

```
1 if (text[k + j] != query[k]) {  
2     match = false;  
3     break;  
4 }
```

### 3.4 Evaluation

## 4 Summary

## References

- [1] [www.geeksforgeeks.org](http://www.geeksforgeeks.org). Rabin-karp algorithm for pattern searching, Last Updated : 1 Aug, 2025.